

An Efficient Word-level Black-box Adversarial Attack Against Textual Models

Anonymous Authors

Abstract—Deep learning models are intrinsically susceptible to textual adversarial attacks, where the perturbed text can trigger aberrant behaviours. In this paper, we present a novel word-level attack called SCALA: a Synonym-based desCending And repLace-back Ascending mechanism that can efficiently produce lexically, grammatically and semantically correct adversarial examples. The merits of our attacking solution lie in being: (i) effective – it surpasses seven state-of-the-art attacks on five target models in terms of reducing after-attack accuracy; (ii) imperceptible – it keeps a very low word perturbation rate based on the Hamming (L_0 -norm) distance, achieving a better deceptiveness evidenced by human evaluations; (iii) efficient – our tensor-based parallelization strategy ensures the attacking efficiency compared with baselines; (iv) practical – black-box score-based setting ensures that the adversary only needs to query target models for confidence scores; and (v) transferable – our attack shows competitive transferability on the generated adversarial examples. We also provide our code anonymously via <https://anonymous.4open.science/r/word-level-blackbox-attack-284B/> for the purpose of reproducibility.

Index Terms—Model vulnerability, adversarial attacks, black-box setting, word-level perturbations

I. INTRODUCTION

Over the course of the past few decades, deep neural networks (DNNs) have achieved huge success in diverse domains, such as computer vision (CV) [1], [2], speech recognition [3], [4], and natural language processing (NLP) [5], [6]. However, it has been observed that DNNs can be easily deceived by digital covers with modestly modified perturbations, commonly known as adversarial examples [7]. The natural invisibility and imperceptibility of adversarial examples can be a severe threat, for example, when DNNs are used in safety-critical applications [8].

To date, while detecting adversarial inputs for CV has already been intensively explored [9], how to *efficiently* generate adversarial examples in a *black-box* setting for *textual data* is still an under-explored, yet very critical area [10]. Compared to CV settings, the major challenge in textual adversarial attacks lies in two aspects. Firstly, textual data in discrete space cannot be equivalently processed as pixel values in continuous space. For example, even if we can directly apply adversarial image attacks to a continuous word-embedding space, it becomes impossible to map the generated optimal adversarial examples back to the discrete word-level space while retaining semantics [11]. Secondly, textual perturbations tend to be more noticeable to human perception and can be detected by spell or grammar checkers, whereas slight pixel-level modifications on images are invisible to the naked eyes [12].

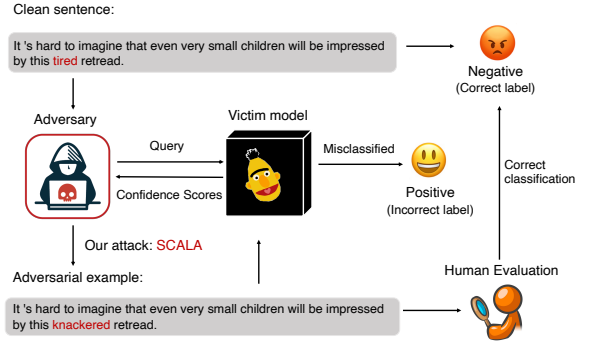


Fig. 1: Illustration of a query-based black-box attack. In sentiment analysis, SCALA generates an adversarial example by querying the victim model (e.g., Bert) to obtain confidence scores and then only substituting one word in the input text with a synonym. This adversarial text can fool the NLP models and evade perception for human evaluators.

In the face of these challenges, the corresponding solutions in existing work for textual adversarial attacks can be categorized as: character-level modifications [13], [14], sentence-level manipulations [15], [16], and word-level perturbations [17], [18]. The first two include inserting, flipping, swapping, and removing characters or sentences [19], [20]. Although those methods aim to introduce visually similar perturbations, they break language specification and textual naturality. Namely, it is valid to defend against such attacks by adopting spell or grammar checkers [21]. In comparison, word-level attacks are usually achieved by inserting, substituting, and removing words [22], [23]. Specifically, synonym substitution can generate semantically similar sentences that preserve grammatical correctness to avoid the above limitations. In this regard, this paper proposes **SCALA**: a **S**ynonym-based **d**esCending **A**nd **r**epLace-back **A**scending strategy. In Fig. 1, an example of our mechanism shows that our attack performs a minor substitution with a synonym, enabling the misclassification of the model in sentiment analysis and bypassing human evaluation. The key contributions of this paper are summarized below.

(i) Our attack based on hamming distance with a flexible and practical *double-sorting* synonym-level mechanism effectively ensures a *small quantity* of modifications and the *semantically similar* adversarial examples. Our attack can work in a black-box setting where only the model’s confidence scores are accessible.

(ii) We evaluate our attack on *five* well-established NLP victim models over multiple text classification and textual entailment tasks. Compared to previous research, it achieves

a *higher* success rate, *lower* perturbation rate, *better* transferability, *satisfactory* grammatical errors under both software checker and human judges.

(iii) We also optimize our black-box attack through *parallelization*, which leads to a *superior* performance in terms of *efficiency*, e.g., it can generate an adversarial sentence within *2.5 seconds* for IMDB dataset, **2 to 1100 times** faster than the baselines.

II. RELATED WORK

We review existing textual adversarial attacks based on their accessibility to model knowledge.

Gradient-based Modification: In this category, the continuous word vector space has been utilized, leveraging gradients generated by a victim model for either attacks or robust training. For example, HotFlip [24] generated adversarial text by computing derivatives with respect to a few character-edit operations (i.e., flip, insert, delete) in its one-hot representation. Textbugger [19] found important words by computing the Jacobian matrix of the classifier and then chose an optimal perturbation from the generated perturbations. However, these attacks require access to detailed information about the target model, which is less realistic in real-world situations [12].

Hard-label (Decision-based) Attacks: This category focuses on the very challenging hard-label setting where the adversary can only access the classification labels of the target model. Recently, [25] has explored this setting using a combination of search space reduction and a genetic algorithm. Decision-based black-box attacks are more restricted and practical than score-based black-box attacks in real-world scenarios. However, attacks under this setting are time-consuming.

Soft-label (Score-based) Attacks: This group belongs to the black-box attack but is less strict than the hard-label setting since the confidence scores of the prediction are available to the adversary. [26] proposed a method to generate semantically and syntactically similar adversarial examples based on a genetic algorithm. [27] incorporated the sememe-based word substitution and particle swarm optimization to solve the combinatorial optimization. PWWS [23] proposed a greedy algorithm called probability-weighted word saliency for textual adversarial attack based on synonym substitution. Similarly, TextFooler [22] generated adversarial examples at the word level that preserve semantics using synonym substitution. BERT Attack [28] used pre-trained masked language models exemplified by BERT to add perturbations. Recently, [29] has jointly leveraged the attention mechanism and locality-sensitive hashing to reduce queries.

Our work belongs to this category. Our soft-label attack ensures a balance between practicality, effectiveness, and efficiency with a novel synonym-based descending and replace-back ascending mechanism to sort synonyms and reduce redundant substitution for producing less perceptible textual adversarial examples.

III. METHODOLOGY

A. Problem Formulation

Given an input sentence X with n words from a corpus dataset. Let $X = \{x_i \mid i \in \mathbb{N}_{i \leq n}^*\}$, where x_i denotes the i th word in the original sentence. A target model f is pre-trained to learn a mapping from the input X to a prediction label Y . Specifically, the victim model f computes confidence scores $C(X)$ for the input. Thus, the final classification label is represented by $Y = cl(f, X) = \operatorname{argmax}_{l \in \{1, \dots, m\}} C_l(X)$ with the highest confidence score $C_l(X)$ selected from m labels. In our score-based black-box setting, the adversary is unaware of the model architecture, parameter settings, or training data, and can only query the target model with supplied inputs to get the output prediction scores.

Norm Selection: L_0 Norm. We denote the distance metric by $\|\cdot\|_D$. In our strategy, L_0 norm is adopted, i.e., Hamming distance. Given an original sentence X and a corresponding adversarial sentence X_{adv} , the L_0 distance is denoted as $\|X_{adv} - X\|_0$, which measures the number of the words that are different. Intuitively, a smaller L_0 distance implies an adversarial example closer to the original sentence and is harder to be discerned by software tools and human evaluation.

Constraint-optimization Problem. For a given input X , we aim to craft an adversarial example X_{adv} by word-level synonym substitution. Specifically, the generation of a valid adversarial sequence X_{adv} should conform to the requirement:

$$\min_{X_{adv}} \|X_{adv} - X\|_0 \text{ s.t. } cl(f, X) \neq cl(f, X_{adv}) \quad (1)$$

where the objective is to optimize an adversarial sentence X_{adv} such that it has a minimum Hamming distance with the clean sentence X . In the meantime, the adversarial text X_{adv} can mislead the NLP classifier to make an incorrect prediction.

However, noted that the problem defined here is NP-hard owing to complexity and non-convex of NLP models [30]. Our strategy proposes an approximation-based solution, which aims to limit the perturbation quantity on a small scale and simultaneously guarantees high efficiency.

B. SCALA: a Synonym-based desCending And repLace-back Ascending Mechanism

In this subsection, the design of SCALA is introduced, here an overview is first provided. As seen in **Fig. 2**, four steps constitute the generation of an adversarial sentence. **Step-1** filters out part-of-speech mismatched and stop words in an original input X and obtains a set Q_{sent} with sentence-order word indices. Then **Step-2** flexibly samples synonyms to create X_{one} with one-word substituted and resorts a set Q_{rep} with word indices based on saliency scores S_{one} in descending order. **Step-3** iteratively generates X_{temp} with accumulated perturbations based on Q_{rep} until an adversarial sentence X_{adv} is created. Finally, **Step-4** computes saliency scores via one-word replace-back sentences X_{back} and reorders word indices in ascending order. A redundancy-eliminated adversarial sentence X_{adv} is obtained by accumulatively replacing back words. Details are stated as below.

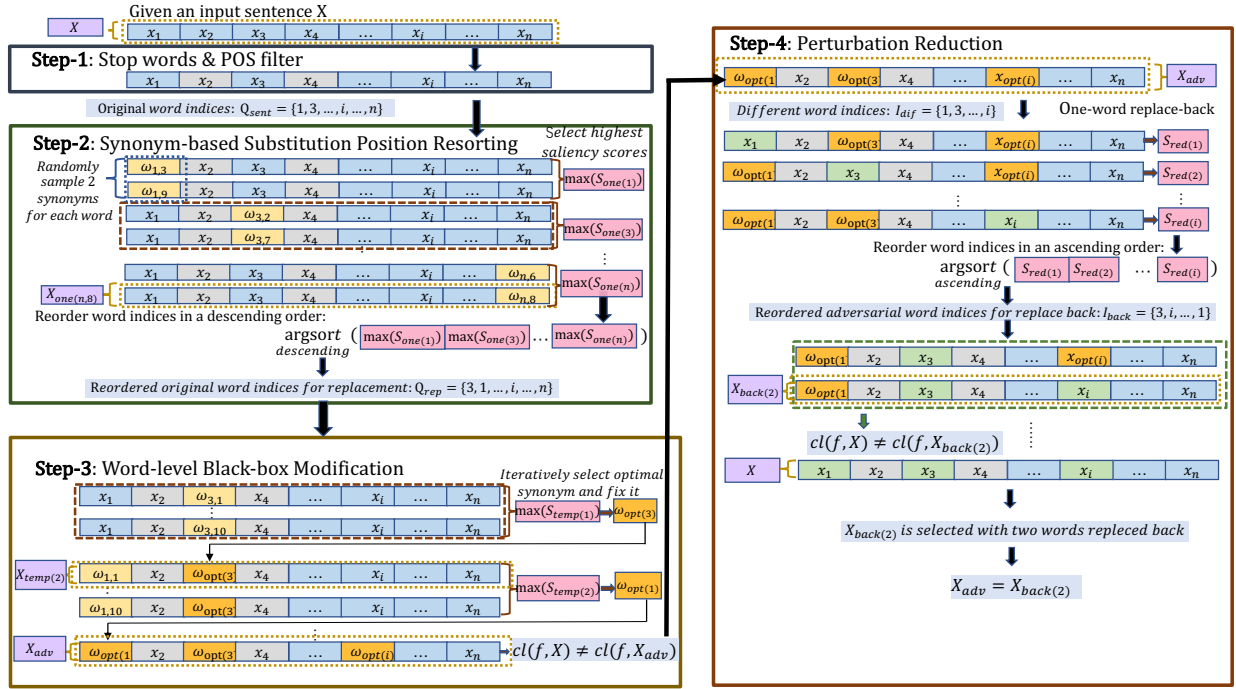


Fig. 2: Illustration for the generation of adversarial text in SCALA. Assume that 2 synonyms are randomly selected for each word, the meaning for colors of blocks in Step-2: **purple**: sentences in different states (X : original, X_{one} : one-word substitution, X_{temp} : accumulated perturbation, X_{adv} : adversarial example and X_{back} : adversarial example after replacing back), **blue**: original words, **grey**: filtered words, **yellow**: candidate synonyms, **orange**: local optimal synonyms, **pink**: saliency scores, **green**: replace-back words.

Step-1: Filter Rules and Synonym Selection

We first filter out meaningless words and select synonym candidates in this step. The set of stop words, originating from NLTK2¹ and spaCy3² libraries, is adopted as the strategy of filtering out word positions to be replaced. Given an input X with n words, a set of eligible word indices for substitution after filtering is denoted as $Q_{sent} \subseteq \{1, \dots, n\}$. Meaningless words such as articles, prepositions and numbers are filtered out in this stage, guaranteeing the syntactic fluency and grammatical correctness for perturbed sequences.

Following this, synonym selection is performed. We select a set of candidate synonyms for each substitution position in Q_{sent} via the counter-fitted embedding space [31]. Then, considering an original word x_i , $Z_{syn(i)}$ is represented as a set including all the synonym candidates for x_i :

$$Z_{syn(i)} = \{\omega_{i,j} \mid i \in Q_{sent} \wedge \forall j \in \mathbb{N}_{j \leq |Z_{syn(i)}|}^*\} \quad (2)$$

where $\omega_{i,j}$ is a single synonym among the set $Z_{syn(i)}$, i denotes the index in the set Q_{sent} with original word indices and j denotes the index in the synonym set $Z_{syn(i)}$.

Step-2: Synonym-based Substitution Position Resorting

In this step, we aim to reasonably sort candidate positions for substitution in the original sentence, a synonym-based reorder strategy for the set Q_{sent} with sentence-order indices is provided. To ensure the higher efficiency especially for datasets with over-length sentences like IMDB, we also introduce a random sampling mechanism to sample the candidate synonyms for each word x_i . Specifically, given a tunable parameter τ , a set

of replacement indices Q_{sent} , the synonym set $|Z_{syn(i)}| = z$, a randomized sampling-based synonym set $Z_{ran(i)}$ for original word x_i is defined as below:

$$Z_{ran(i)} = \{\omega_{i,j} \mid i \in Q_{sent}, j \in \text{random } Z_{syn(i)}\}_{\tau * z} \quad (3)$$

where a randomly selected synonym $\omega_{i,j} \in Z_{syn(i)}$. The function random is utilized to randomly select the fixed number of $|Z_{ran(i)}| = \tau * z$ synonyms. For example, assume that each original word has $z = 10$ candidate synonyms, let $\tau = 0.2$, then 2 candidates are randomly selected for each eligible substitution position. On this basis, a sentence X_{one} with one-synonym perturbation is constructed based on the original sentence X as below:

$$X_{one(i,j)} = \{x_1, x_2, \dots, \omega_{i,j}, \dots, x_n\} \quad (4)$$

where only one original word x_i is replaced with a synonym $\omega_{i,j} \in Z_{ran(i)}$ (3), the original word index $i \in Q_{sent}$ and synonym index $j \in \mathbb{N}_{j \leq |Z_{ran(i)}|}$. Following this, the saliency score $S_{one(i,j)}$ between an input X and a one-word substitution sentence $X_{one(i,j)}$ is computed as:

$$S_{one(i,j)} = C_Y^*(X) - C_Y^*(X_{one(i,j)}) + \mathbb{I}(Y \neq Y^*) * (C_Y(X_{one(i,j)}) - C_Y(X)) \quad (5)$$

where C returns the confidence score under the prediction label. The Boolean expression $\mathbb{I}(Y \neq Y^*)$ equals to 1, If the label for $X_{one(i,j)}$ is different from the input X , i.e., $Y \neq Y^*$. Otherwise, the expression is equal to 0 when the label remains the same. Note that a higher saliency score here indicates a more competitive candidate synonym that results in misclassification for victim models. On the contrary, a lower or even minus saliency score represents limited or even adverse influence on

¹<https://www.nltk.org/>

²<https://github.com/explosion/spaCy>

fooling deep classifiers. Therefore, if i th word x_i being replaced by j th synonym $\omega_{i,j}$ in $Z_{ran(i)}$ causes the most significant change in the classification probability, an optimal synonym $\omega_{opt(i)} = \omega_{i,j}$ is selected for substitution. This created sentence is denoted as $X_{opt(i)}$. The local optimal saliency score $S_{opt(i)}$ is defined as: $\exists X_{opt(i)} \text{ s.t. } S_{opt(i)} = C_Y(X) - C_Y(X_{opt(i)})$.

Furthermore, a merging set S_{opt} made up of individual optimal saliency score $S_{opt(i)}$ is obtained: $S_{opt} = \{S_{opt(i)} \mid \forall i \in Q_{sent}\}$. Thus, the descending order for saliency scores helps construct a reasonable sort Q_{rep} for word replacement by reordering indices in sentence-order set Q_{sent} as follows:

$$Q_{rep} = \left\{ Q_{sent(i)} \mid i \in \underset{\text{descending}}{\text{argsort}} S_{opt} \right\} \quad (6)$$

Step-3: Word-level Adversarial Example Generation

In this step, a word-level modification mechanism is designed based on the reasonable sort Q_{rep} in (6). $X_{temp(k)}$ is created as a sentence that iteratively constructs the i -th word substitution in k th iteration such that $k \leq |Q_{rep}|$. The substitution is realized with a local optimal synonym $\omega_{opt(i,j)} \in Z_{syn(i)}$ defined in (2), which results in the highest saliency score in each iteration. Noted that this word substitution is fixed, and the next substitution is continued based on this fixed sequence until the target model makes an incorrect prediction. Thus, the sentence $X_{temp(k)}$ with accumulated perturbations in k th iteration is denoted as below:

$$X_{temp(k)} = \{(X_{temp(k-1)} \setminus x_i) \cup \omega_{opt(i)} \mid i = Q_{rep(k)}\} \quad (7)$$

where the sentence $X_{temp(k-1)}$ is generated in the last iteration. We replace the original word x_i with the local optimal synonym $\omega_{opt(i)}$, noted that here i means the k th index in the set Q_{rep} with original word indices.

Then the target model is queried with $X_{temp(k)}$ for the confidence score and label. The query is continues until a model misclassification appears. The obtained adversarial sentence is denoted as X_{adv} and satisfies the condition of misleading the NLP model: $\exists X_{adv} \text{ s.t. } cl(f, X) \neq cl(f, X_{adv})$. Assume that in k^* th iteration, this adversarial example is generated. Intuitively, compared with the input X with n words, the adversarial sentence has modified word indices: $\{Q_{rep(1)}, \dots, Q_{rep(k^*)}\} \subseteq Q_{rep}$.

Step-4: Perturbation Reduction for Textual Redundancies

After crafting the adversarial example, it is worth noting that not all the modifications in X_{adv} are required to make an incorrect prediction. Thus, the elimination of the redundancies leads to perturbation reduction, which aims to shrink the Hamming distance to ensure that the adversarial text is more imperceptible to human beings. Assume that the set for different word indices between the adversarial sentence X_{adv} and the input X is denoted as I_{dif} . Similar to vulnerable word sorting in step-2, a different word in adversarial sentence X_{adv} is replaced back to an original word. This one-word replace-back sentence is denoted as $X_{red(i)}$:

$$X_{red(i)} = \{(X_{adv} \setminus x_{adv(i)}) \cup x_i \mid i \in I_{dif}\} \quad (8)$$

where the word $x_{adv(i)}$ in X_{adv} is replaced back to the original word x_i and i is an index in the set I_{dif} . Similar with (5), the saliency score between X_{adv} and $X_{red(i)}$ is computed:

$$S_{red(i)} = C_Y^*(X_{adv}) - C_Y^*(X_{red(i)}) + \mathbb{I}(Y \neq Y^*) * (C_Y(X_{red(i)}) - C_Y(X_{adv})) \quad (9)$$

where $i \in I_{dif}$. Here a lower saliency score indicates a weaker influence on misleading victim models and corresponds to a more appropriate word to be substituted back. The saliency scores $S_{red(i)}$ constitute a set $S_{red} \in \mathbb{R}^{|I_{dif}|}$.

Thus the indices of replace-back words are reordered and a reasonable sort I_{back} in ascending order for perturbation redundancy is obtained:

$$I_{back} = \left\{ I_{dif(i)} \mid i \in \underset{\text{ascending}}{\text{argsort}} S_{red} \right\} \quad (10)$$

Based on the ascending sort I_{back} , the perturbed words in X_{adv} are iteratively substituted back to the original word. The replace-back words are fixed until the last adversarial word is returned. Assume that $X_{back(i)}$ is the sentence with i adversarial words replaced back to original ones based on the set of reordered indices $\{I_{back(1)}, \dots, I_{back(i)}\}$ in i th iteration. Following this, check whether $cl(f, X_{back(i)}) \neq cl(f, X)$. If any element replacement associated with $X_{back(i)}$ leads to true, we conclude that the corresponding words are unnecessary for misclassification. Thus, the last sentence $X_{back(k)}$ is selected as the final adversarial example if it can result in misprediction with as many replace-back words as possible in the ascending order I_{back} . The indices for substituting back are $\{I_{back(1)}, \dots, I_{back(k)}\}$. Conversely, if all the sentences X_{back} are false for $cl(f, X_{back(k)}) \neq cl(f, X)$, we reserve all the modifications for X_{adv} . Consequently, an adversarial sentence X_{adv} with a small quantity of alterations is created.

C. Parallelization Module Design

Due to the inefficiency of computing the saliency scores sequentially when the number of synonyms is high, or massive amounts of data in the dataset, we further improve our algorithm to be more applicable to efficiency-demanding scenarios. Therefore, we utilize tensor-based parallelization to compute the saliency scores simultaneously.

In the aforementioned steps, the set of sentences and the array of importance scores are regarded as integration to process. Specifically, the one-word synonym-replacement sentence X_{one} (4), the potential adversarial example X_{temp} (7), one-word replace-back sentence X_{red} (8) and iterative replace-back sentence X_{back} are combined as a whole list respectively to feed into the NLP classifier to obtain the confidence scores. Following this, importance scores ($S_{one(i,j)}$ (5) for one-word substitution and $S_{red(i)}$ (9) for one-word replace-back) are further computed as multi-dimensional arrays parallelly. The parallelization operations enable better efficiency in dealing with high volumes of data.

IV. EXPERIMENTS

A. Experimental Settings

Datasets As illustrated in Table I, there are overall four datasets for text classification and two datasets for textual

TABLE I: Statistics of all datasets. **Train:** the number of training dataset, **Test:** the number of test dataset, **Avg. Len.:** the average length of sentences, **Lab.:** the number of classification labels.

Task	Dataset	Train	Test	Avg. Len.	Lab.
Text Classification	MR	9K	1K	20	2
	IMDB	25K	25K	215	2
	Yelp	560K	38K	152	2
	AG	120K	7.6K	43	4
Textual	SNLI	570K	3K	8	3
Entailment	MultiNLI	433K	10K	11	3

entailment. Table I has provided statistics of sample size for the training and testing process, the average length for each sentence and the number of prediction labels³. For **text classification**: (i) **MR**: The dataset contains both positive and negative sentences extracted from Rotten Tomatoes movie reviews [32]⁴. (ii) **IMDB**: As a dataset for binary sentiment classification, a set of highly polar movie reviews for testing and training are provided [33]⁵. (iii) **Yelp**: A classification task is constructed from this dataset, using stars 1 and 2 as negative and 3 and 4 as positive for predicting polarity labels [34]⁶. (iv) **AG**: Based solely on the title and description fields of news articles, 4 largest classes are selected from this corpus to construct the dataset [34]⁷. For **textual entailment**: (i) **SNLI**: It contains a collection of pairs of human-written English sentences, which aims to assess whether two sentences are entailment, contradiction, or neutral [35]⁸. (ii) **MultiNLI**: This corpus is a crowd-sourced set of sentence pairs, it covers the genre-diversity of spoken and written sentences [36]⁹.

Victim Models As part of our textual classification task, we conducted attacks on target models as follows. (i) **WordCNN**: A total of 150 filters were used in WordCNN in windows of sizes 3, 4 and 5 [37]. (ii) **WordLSTM**: A single layer bi-direction LSTM with 150 hidden units was used. (iii) **BERT Base-uncased**: Using the 12-layer BERT model, we included 768 hidden units and 12 heads, with 110M parameters, known as the base-uncased version [38]¹⁰. Regarding textual entailment, BERT base-uncased is utilized as well. Additionally, two more models are also adopted: (i) **ESIM**: An enhanced model for natural language inference based on chain LSTM [39]¹¹. (ii) **InferSent**: An English sentence encoder for universal sentence embeddings, which is trained using the supervised data of SNLI dataset [40]¹².

Baselines We have selected seven representative baselines for evaluating the performance of our attack. The descriptions of the baselines are stated below. (i) **Genetic Attack**: A black-box

attack adopting population-based optimization algorithm to generate imperceptible adversarial examples [26]. (ii) **TextBugger**: In the black-box scenario, it finds important sentences and then uses a scoring function to determine which words are important to manipulate. [19]. (iii) **TextFooler**: This work ranks words based on important scores and considers sentence similarity scores to craft attacks [22], we compare this attack under both tasks of text classification and entailment. (iv) **PSO**: Sememe-based word substitution method and adopted particle swarm optimization-based algorithm technique [27]. (v) **PWWS**: This attack determines the order of word substitution is based on word saliency and weighted by classification probability [23]. (vi) **Bert-Attack**: As a perturbation generator, it utilizes the masked language model to find perturbations that increase the risk of making wrong predictions for target models [28]. BAE is not on the list since it performs similarly to BERT-Attack [41]. (vii) **Hard-label Attack**: It adopted a genetic algorithm to generate adversarial examples in a hard-label setting [25].

Parameter Settings In our experiments, a NLTK¹³ filter is used to filter stop words, while Spacy¹⁴ tags are used for POS tags. In accordance with the procedures discussed by [26], we test our algorithm with randomly selected 1,000 examples from the test set in each dataset. The number z of synonyms for each word is set to **50** and the parameter τ in (3) is set to **0.06** and **0.2** for long-length datasets IMDB and Yelp respectively. For short-sentence datasets, $\tau = 1$.

Evaluation Metrics: (i) **After-attack Accuracy**: It refers to the accuracy of the target model based on generated adversarial examples. (ii) **Perturbation Rate**: The percentage of the number of words substituted in input for creating an adversarial example. (iii) **Grammatical Correctness**: We track the grammatical error rate changes between the original and final adversarial samples via Language-Tool¹⁵. (iv) **Human Evaluation**: Human evaluation was also conducted to ensure that the generated adversarial examples are of a high standard under the human perception, including successful prediction accuracy, grammatical correctness assessment and semantic similarity. The human evaluation comprises three criteria: classification accuracy, grammatical correctness, and semantic similarity. The BERT model was targeted on the MR and SNLI datasets using 200 randomly selected test sentences to generate adversarial examples. Using the same operations as [42], we mixed original and adversarial texts and allocate human judges to evaluate their grammatical accuracy and semantic similarity on a Likert scale of 1-5, following the practice as [22]. Before annotation, candidates underwent a test. To determine whether our results are natural, we ask five humans to assign subcategories to each of our examples while true labels are hidden and compute the average rates and scores.

³For the purpose of reproducibility, our code is anonymously released via <https://anonymous.4open.science/r/word-level-blackbox-attack-284B/>

⁴<https://www.cs.cornell.edu/people/pabo/movie-review-data/>

⁵<https://datasets.imdbws.com/>

⁶<https://www.yelp.com/dataset>

⁷http://groups.di.unipi.it/~gulli/AG_corpus_of_news_articles.html

⁸<https://nlp.stanford.edu/projects/snli/>

⁹<https://cims.nyu.edu/~showman/multinli/>

¹⁰<https://github.com/huggingface/pytorch-pretrained-BERT>

¹¹<https://github.com/coetaur0/ESIM>

¹²<https://github.com/facebookresearch/InferSent>

¹³<https://www.nltk.org/>

¹⁴<https://github.com/explosion/spaCy>

¹⁵<https://languagetool.org/>

TABLE II: Task of text classification in contrast with 7 baselines (**2nd column**) over 4 datasets (**1st column**) for 3 classifiers (**1st row**). The abbreviations: original accuracy(**Orig.%**), after attack accuracy(**Acc.%**) and perturbation rate (**Pert.%**). ↓ represents that the lower the better.

Dataset	Attack	Bert				WordLSTM				WordCNN			
		Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓
MR	GA	96.90%	32.00%	17.39%	12.68%	86.60%	17.00%	18.39%	13.81%	85.40%	11.00%	18.49%	11.68%
	TextBugger		35.30%	13.89%	35.72%		18.40%	13.20%	27.53%		18.80%	14.41%	34.92%
	Hard-label Attack		2.00%	13.34%	6.36%		2.70%	13.58%	4.07%		2.30%	12.02%	8.07%
	PSO		6.00%	20.21%	3.58%		4.30%	14.27%	2.23%		4.20%	15.47%	4.80%
	BERT-Attack		23.22%	16.09%	13.51%		8.80%	16.53%	13.62%		10.10%	17.29%	16.55%
	PWWS		13.70%	15.20%	6.29%		5.60%	12.86%	6.99%		4.30%	13.24%	8.73%
	TextFooler		7.10%	18.57%	9.17%		1.00%	13.08%	5.79%		0.50%	14.06%	7.16%
	Ours		2.10%	10.39%	4.44%		0.20%	9.32%	3.49%		0.00%	9.70%	6.33%
IMDB	GA	89.10%	46.00%	5.31%	1.93%	89.20%	13.00%	5.23%	1.88%	88.10%	6.00%	5.26%	1.73%
	TextBugger		11.00%	8.68%	11.78%		0.40%	4.33%	4.56%		0.20%	4.06%	4.88%
	Hard-label Attack		1.00%	3.65%	2.20%		1.00%	3.27%	1.74%		0.00%	3.32%	2.33%
	PSO		1.30%	3.70%	2.00%		0.00%	2.76%	0.39%		0.00%	3.01%	0.61%
	BERT-Attack		9.50%	5.55%	1.99%		0.70%	4.73%	1.01%		0.10%	4.19%	1.17%
	PWWS		2.50%	4.68%	1.46%		0.20%	2.33%	0.75%		0.00%	2.04%	0.74%
	TextFooler		1.30%	8.54%	3.11%		0.00%	2.69%	0.93%		0.00%	2.46%	0.92%
	Ours		0.00%	2.28%	1.07%		0.00%	1.55%	0.96%		0.00%	1.56%	1.37%
Yelp	GA	94.60%	24.00%	11.34%	5.62%	94.00%	5.00%	7.72%	3.02%	94.00%	18.00%	7.52%	2.98%
	TextBugger		14.60%	10.43%	13.54%		4.40%	8.02%	7.06%		5.80%	8.86%	8.98%
	Hard-label Attack		5.00%	6.37%	5.16%		3.00%	5.19%	2.77%		1.00%	6.12%	4.09%
	PSO		1.00%	11.04%	0.44%		0.00%	7.07%	0.70%		1.00%	7.48%	0.35%
	BERT-Attack		7.91%	8.09%	3.14%		2.40%	8.62%	1.44%		3.60%	8.85%	1.66%
	PWWS		6.90%	6.92%	1.91%		0.90%	4.97%	1.01%		1.10%	5.73%	1.28%
	TextFooler		5.80%	9.76%	3.23%		0.30%	5.61%	1.44%		0.40%	6.38%	1.91%
	Ours		1.00%	4.43%	1.78%		0.00%	3.60%	1.68%		0.20%	4.34%	2.58%
AG	GA	94.60%	46.00%	16.55%	7.09%	89.90%	41.00%	15.46%	6.89%	90.70%	39.00%	18.19%	6.94%
	TextBugger		42.00%	18.58%	26.84%		22.30%	19.00%	20.91%		11.80%	17.27%	20.84%
	Hard-label Attack		5.80%	12.20%	5.50%		5.70%	13.20%	4.08%		1.40%	10.54%	5.63%
	PSO		20.60%	16.70%	1.46%		10.90%	20.45%	2.45%		8.10%	20.03%	2.26%
	BERT-Attack		35.13%	17.97%	0.61%		31.60%	18.31%	0.95%		17.70%	19.80%	2.07%
	PWWS		37.80%	17.20%	2.44%		14.60%	16.59%	3.62%		7.80%	14.54%	4.65%
	TextFooler		16.10%	22.82%	8.00%		4.70%	17.92%	6.74%		0.70%	15.37%	6.06%
	Ours		4.30%	10.17%	2.38%		0.90%	10.62%	6.44%		0.00%	7.86%	5.73%

TABLE III: Task of text entailment in contrast with TextFooler (TF). The abbreviations are represented as original accuracy(**Orig.%**), after attack accuracy(**Acc.%**) and perturbation rate (**Pert.%**). ↓ denotes that the lower the better.

Dataset	Attack	Bert				InferSent				ESIM			
		Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓	Orig.%	Acc.% ↓	Pert.% ↓	Gram.% ↓
SNLI	TF	89.1%	1.70%	7.18%	36.4%	79.0%	6.50%	5.86%	31.3%	86.5%	5.10%	9.05%	25.5%
	Ours		0.40%	5.36%	17.2%		1.70%	5.74%	11.0%		1.10%	5.42%	8.8%
MultiNLI (matched)	TF	85.1%	5.30%	7.03%	29.4%	67.3%	8.00%	5.32%	17.3%	78.1%	7.70%	7.25%	20.1%
	Ours		0.50%	4.68%	10.8%		0.70%	5.11%	9.90%		0.50%	4.40%	9.30%
MultiNLI (mismatched)	TF	85.1%	3.40%	6.68%	28.90%	62.4%	6.90%	5.74%	22.10%	75.5%	7.30%	7.30%	22.40%
	Ours		0.30%	4.43%	11.5%		0.40%	4.69%	8.70%		0.10%	4.20%	11.40%

B. Analysis of Attack Performance Compared with State-of-the-art and Software Evaluation

In Table II and Table III, the experimental results show that our attack achieves overall better performance in terms of **after-attack accuracy**, **average perturbation rate** and the **increasing rate of grammatical errors** for text classification and text entailment. While some work filters out adversarial examples with high perturbations, we measure all the perturbed sentences without any filtering to avoid a biased view. **After-attack accuracy** and **perturbation rate** are reduced to below 5% and 10% respectively. It is worth noting that three models are totally misled on IMDB dataset with 215 words on average (reaching the accuracy of 0%) with a very low word perturbation rate (2.28%, 1.55% and 1.56% respectively). Therefore, our attack is proved to perform less modifications with lower after-attack accuracy. The **software evaluation** of increasing rate of grammatical errors between the original and adversarial samples shows that our performance under this evaluation is satisfactory with a low increasing rate of grammatical errors. Although our rate is not always the lowest,

it is normal to appear some grammatical errors in a real-life scenario and can deceive the naked eyes of human beings. Therefore, our results achieve satisfactory performance.

Noted that **compared with PWWS**, our method has only 10.17% words are modified on AG dataset for attacking state-of-the-art BERT which reduces the accuracy from 94.6% to 4.3% after-attack accuracy. While PWWS has perturbed 17.2% words with high after-attack accuracy of 37.8%. Although PWWS also computed saliency scores for word-level substitution, the strategy of replacing unknown word (out of vocabulary) for importance sorting could easily break the naturality of sentences and greatly impact operations afterwards. Moreover, a replace-back stage gives our method an edge in further reducing perturbation redundancy. Additionally, the efficiency analysis in next subsection also proves our competitive runtime capacity.

TABLE IV: Comparison with population-based attacks.

Dataset	Method	Runtime(s/sentence)
IMDB	GA	2835.32
	Hard-label Attack	458.90
	PSO	1203.37
	Ours (Parallel)	2.56

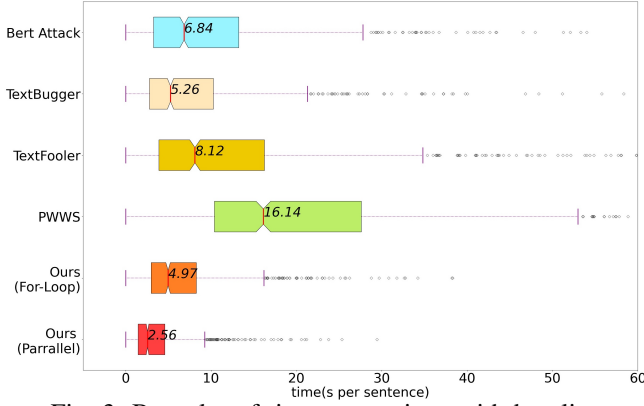


Fig. 3: Box plot of time comparison with baselines

TABLE V: Transferability comparison after-attack accuracy with TextFooler(TF) on IMDB dataset. Models listed on the first column create the adversarial samples and then test on models list on the first row.

Gen.	Test	Attack	wordCNN	wordLSTM	BERT
wordCNN		TF	-	84.9%	90.20%
		Ours	-	18.02%	4.01%
wordLSTM		TF	74.9%	-	87.9%
		Ours	34.80%	-	10.09%
BERT		TF	84.1%	85.1%	-
		Ours	12.2%	11.4%	-

C. Ablation Study on Computational Efficiency

The runtime of our attack is competitive in contrast to prior work on IMDB dataset against BERT model. In **Fig. 3**, our parallel-based attack shows 2-6 times faster than other baselines for per-sentence runtime, while our for-loop strategy is also competitive in terms of efficiency. Furthermore, our parallel-efficient attack achieves 170-1100 times dramatically higher speed than population-based attacks in **Table IV**. We perform the computation on a PC with i7-8559U CPU, 16GB of RAM and NVIDIA GeForce RTX 2080 Ti GPU.

D. Adversarial Transferability and Adversarial Training

To study the transferability, we select generated adversarial examples that were incorrectly classified by a given target model and reevaluate the classification accuracy of those examples for attacking other models. The test sets are collected from IMDB and SNLI for both tasks of text classification and textual entailment. The results of after-attack accuracy are shown in **Table V** and **Table VI**. It is worth noting that higher transferability is correlated with lower after-attack accuracy.

TABLE VI: Transferability comparison with TextFooler(TF) on SNLI dataset. Models listed on the first column create the adversarial samples and then test on model list on the first row.

Gen. \ Test	Attack	BERT	InferSent	ESIM
BERT	TF	-	58.2%	54.6%
	Ours	-	58.5%	61.1%
InferSent	TF	67.7%	-	62.7%
	Ours	17.6%	-	20.5%
ESIM	TF	59.3%	49.4%	-
	Ours	16.9%	19.4%	-

TABLE VII: Comparison of the original accuracy ("Orig.%"), after-attack accuracy ("Acc.%"), average percentage of perturbed words ("Pert.%") and the number of queries(Que.) of original training ("Orig.") and adversarial training ("Adv.") of BERT on MR dataset.

Dataset	Train	Bert			
		Orig.%	Acc.%	Pert.%	Que.
MR	Orig.	92.2	10.1	12.8	155
	Adv.	85.6	15.5	13.938	165.1

TABLE VIII: Human-evaluation results show the comparison of the original (**Orig.**) and adversarial (**Adv.**) sentences in terms of model prediction accuracy (**Acc.**), grammatical correctness (**Gra.**) and semantic similarity (**Sem.**) of BERT model on MR and SNLI datasets.

Dataset		Acc.	Gra.	Sem.
MR	Orig.	96.5%	4.92	4.18
	Adv.	83.0%	4.77	
SNLI	Orig.	91.0%	4.94	4.01
	Adv.	64.5%	4.52	

Compared with the baseline TextFooler, our attack shows considerably better transferability over multiple datasets.

Regarding the generated adversarial examples, our research sheds light on how to better improve the robustness of the target models. We obtained all the adversarial examples that deceived BERT from the MR training sets and mixed them with the original training set. During an initial trial of adversarial training, we fed the models both clean sentences and adversarial examples. The labels of generated adversarial samples are processed back to their initial counterparts for observing whether this operation can contribute to strengthening model robustness. Our next step was to train BERT with the augmented data. Following this, adopting the original dataset MR to attack this retrained model and assess it via recalculating after-attack accuracy, perturbation rate and average queries, which shows increases of 5.4%, 1.2% and 10 respectively as shown in **Table VII**. It comes as no surprise that adding adversarial samples to the training data improves the target model's robustness to attacks, since it then requires more perturbations and queries.

E. Human Evaluation

We conducted a human evaluation of the generated adversarial samples to determine the quality of the generated samples in terms of *classification accuracy*, *grammatical correctness*, and shared *semantic similarity* with original sentences. As seen in **Table VIII**, the successful classification accuracy for adversarial samples is close to the original accuracy for MR dataset. For SNLI dataset, the gap of accuracy is mainly due to the short length of the hypothesis sentences that restrict the synonym substitution space. The grammatical correctness and semantic similarity for both tasks are relatively high, owing to synonym substitution and POS filter.

V. CONCLUSION

For black-box adversarial attacks, to preserve the human imperceptibility, adversarial texts are either visually or seman-

tically similar to original sentences. By observing only the topmost predicted labels and scores of adversarial examples, we develop a novel score-based attack that employs a parallel-efficient algorithm to generate semantically similar adversarial examples. The efficacy and effectiveness of our proposed attack have been demonstrated on a wide range of target models and benchmark datasets on several competitive baselines. Compared to prior attack strategies, our approach has higher success rates, lower perturbation rates. Both the human and machine evaluation confirmed that the generated adversarial texts were grammatically correct and similar to the originals.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [2] X. Feng, Y. Jiang, X. Yang, M. Du, and X. Li, "Computer vision algorithms and hardware implementations: A survey," *Integration*, vol. 69, pp. 309–320, 2019.
- [3] B. Bharathi, B. R. Chakravarthi, S. Cn, N. Sripriya, A. Pandian, and S. Valli, "Findings of the shared task on speech recognition for vulnerable individuals in tamil," in *Proceedings of the Second Workshop on Language Technology for Equality, Diversity and Inclusion*, 2022, pp. 339–345.
- [4] O. Abdel-Hamid, A.-r. Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Transactions on audio, speech, and language processing*, vol. 22, no. 10, pp. 1533–1545, 2014.
- [5] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio, "Generating sentences from a continuous space," *arXiv preprint arXiv:1511.06349*, 2015.
- [6] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," *arXiv preprint arXiv:1906.02243*, 2019.
- [7] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.
- [8] X. Huang, D. Kroening, W. Ruan, J. Sharp, Y. Sun, E. Thamo, M. Wu, and X. Yi, "A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability," *Computer Science Review*, vol. 37, p. 100270, 2020.
- [9] T. Bai, J. Luo, J. Zhao, B. Wen, and Q. Wang, "Recent advances in adversarial training for adversarial robustness," *arXiv preprint arXiv:2102.01356*, 2021.
- [10] W. Wang, R. Wang, L. Wang, Z. Wang, and A. Ye, "Towards a robust deep neural network in texts: A survey," *arXiv preprint arXiv:1902.07285*, 2019.
- [11] W. Y. Wang, J. Li, and X. He, "Deep reinforcement learning for nlp," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts*, 2018, pp. 19–21.
- [12] W. E. Zhang, Q. Z. Sheng, A. Alhazmi, and C. Li, "Adversarial attacks on deep-learning models in natural language processing: A survey," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 11, no. 3, pp. 1–41, 2020.
- [13] J. Ebrahimi, D. Lowd, and D. Dou, "On adversarial examples for character-level neural machine translation," *arXiv preprint arXiv:1806.09030*, 2018.
- [14] S. Eger, G. G. Şahin, A. Rücklé, J.-U. Lee, C. Schulz, M. Mesgar, K. Swarnkar, E. Simpson, and I. Gurevych, "Text processing like humans do: Visually attacking and shielding nlp systems," *arXiv preprint arXiv:1903.11508*, 2019.
- [15] Y. Xu, X. Zhong, A. J. Yepes, and J. H. Lau, "Grey-box adversarial attack and defence for sentiment classification," *arXiv preprint arXiv:2103.11576*, 2021.
- [16] T. Le, S. Wang, and D. Lee, "Malcom: Generating malicious comments to attack neural fake news detection models," in *2020 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2020, pp. 282–291.
- [17] D. Emelin, I. Titov, and R. Sennrich, "Detecting word sense disambiguation biases in machine translation for model-agnostic adversarial attacks," *arXiv preprint arXiv:2011.01846*, 2020.
- [18] B. Liang, H. Li, M. Su, P. Bian, X. Li, and W. Shi, "Deep text classification can be fooled," *arXiv preprint arXiv:1704.08006*, 2017.
- [19] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "Textbugger: Generating adversarial text against real-world applications," *arXiv preprint arXiv:1812.05271*, 2018.
- [20] W. Han, L. Zhang, Y. Jiang, and K. Tu, "Adversarial attack and defense of structured prediction models," *arXiv preprint arXiv:2010.01610*, 2020.
- [21] D. Pruthi, B. Dhingra, and Z. C. Lipton, "Combating adversarial misspellings with robust word recognition," *arXiv preprint arXiv:1905.11268*, 2019.
- [22] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is bert really robust? natural language attack on text classification and entailment," *arXiv preprint arXiv:1907.11932*, vol. 2, 2019.
- [23] S. Ren, Y. Deng, K. He, and W. Che, "Generating natural language adversarial examples through probability weighted word saliency," in *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019, pp. 1085–1097.
- [24] J. Ebrahimi, A. Rao, D. Lowd, and D. Dou, "Hotflip: White-box adversarial examples for text classification," *arXiv preprint arXiv:1712.06751*, 2017.
- [25] R. Maheshwary, S. Maheshwary, and V. Pudi, "Generating natural language attacks in a hard label black box setting," in *Proceedings of the 35th AAAI Conference on Artificial Intelligence*, 2021.
- [26] M. Alzantot, Y. Sharma, A. Elgohary, B.-J. Ho, M. Srivastava, and K.-W. Chang, "Generating natural language adversarial examples," *arXiv preprint arXiv:1804.07998*, 2018.
- [27] Y. Zang, F. Qi, C. Yang, Z. Liu, M. Zhang, Q. Liu, and M. Sun, "Word-level textual adversarial attacking as combinatorial optimization," *arXiv preprint arXiv:1910.12196*, 2019.
- [28] L. Li, R. Ma, Q. Guo, X. Xue, and X. Qiu, "Bert-attack: Adversarial attack against bert using bert," *arXiv preprint arXiv:2004.09984*, 2020.
- [29] R. Maheshwary, S. Maheshwary, and V. Pudi, "A strong baseline for query efficient attacks in a black box setting," *arXiv preprint arXiv:2109.04775*, 2021.
- [30] X. Dong, A. T. Luu, R. Ji, and H. Liu, "Towards robustness against natural language word substitutions," *arXiv preprint arXiv:2107.13541*, 2021.
- [31] N. Mrkšić, D. O. Séaghdha, B. Thomson, M. Gašić, L. Rojas-Barahona, P.-H. Su, D. Vandyke, T.-H. Wen, and S. Young, "Counter-fitting word vectors to linguistic constraints," *arXiv preprint arXiv:1603.00892*, 2016.
- [32] B. Pang and L. Lee, "Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales," *arXiv preprint cs/0506075*, 2005.
- [33] A. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, 2011, pp. 142–150.
- [34] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," *Advances in neural information processing systems*, vol. 28, pp. 649–657, 2015.
- [35] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," *arXiv preprint arXiv:1508.05326*, 2015.
- [36] A. Williams, N. Nangia, and S. R. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," *arXiv preprint arXiv:1704.05426*, 2017.
- [37] Y. Kim, "Convolutional neural networks for sentence classification," *arXiv preprint arXiv:1408.5882*, 2014.
- [38] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [39] Q. Chen, X. Zhu, Z. Ling, S. Wei, H. Jiang, and D. Inkpen, "Enhanced lstm for natural language inference," *arXiv preprint arXiv:1609.06038*, 2016.
- [40] A. Conneau, D. Kiela, H. Schwenk, L. Barrault, and A. Bordes, "Supervised learning of universal sentence representations from natural language inference data," *arXiv preprint arXiv:1705.02364*, 2017.
- [41] S. Garg and G. Ramakrishnan, "Bae: Bert-based adversarial examples for text classification," *arXiv preprint arXiv:2004.01970*, 2020.
- [42] J. Gagnon-Marchand, H. Sadeghi, M. A. Haidar, and M. Rezagholizadeh, "Salsa-text: self attentive latent space based adversarial text generation," in *Canadian Conference on Artificial Intelligence*. Springer, 2019, pp. 119–131.